

Формальное доказательство корректности фиксированного векторного представления архитектур CNN

Daniil V. Lukin *

* ITMO University, Saint Petersburg, Russia

Abstract: В работе дается формальное обоснование представления, использованного в лабораторной работе 5. Архитектура сверточной нейронной сети моделируется ориентированным ациклическим графом с типизированными вершинами и параметрами. Затем вводятся канонизация графа, сериализация, сжатие и преобразование в вектор из четырех целочисленных компонент. Доказываются детерминированность кодирования, точность round-trip восстановления для корректных графов, корректность процедуры декодирования, невозможность точного кодирования всех архитектур в конечный fixed-bit вектор и объясняется, почему выбранный вектор из четырех целых чисел не противоречит этому ограничению и поддерживает skip connections и ветвления.

Keywords: CNN, граф архитектуры, формальное доказательство, round-trip корректность, Typst

1. ВВЕДЕНИЕ

Цель лабораторной работы 5 состоит в том, чтобы построить векторное представление архитектуры сверточной нейронной сети, получить его из архитектуры и затем восстановить архитектуру по этому представлению. При этом требуется, чтобы:

- вектор имел фиксированное число компонент;
- восстановленная архитектура была корректной;
- метод работал для сетей произвольного размера;
- метод поддерживал не только последовательные сети, но и связи вида ResNet и Inception.

В реализованном решении архитектура представляется ориентированным ациклическим графом, после чего весь канонический граф кодируется в вектор

$$z = [m, v, p, h] \in \mathbb{Z}^4. \quad (1)$$

Здесь m — сигнатура формата, v — версия схемы, p — целочисленный payload, а h — контрольная сумма. Ниже доказываются основные свойства этой конструкции.

2. МАТЕМАТИЧЕСКАЯ МОДЕЛЬ

2.1 Граф архитектуры

Определение 1. Поддерживаемым графом архитектуры называется четверка

$$G = (V, E, \tau, \pi), \quad (2)$$

где:

- V — конечное множество вершин;
- $E \subseteq V \times V$ — множество ориентированных ребер;
- $\tau : V \rightarrow T$ — отображение, которое каждой вершине ставит в соответствие тип узла;
- π — отображение, которое каждой вершине ставит в соответствие конечное множество параметров.

В данной лабораторной поддерживаются типы узлов Input, Conv2d, BatchNorm2d, ReLU, MaxPool, Add, Concat, Flatten, Linear, Output.

Граф считается *корректным*, если выполняются условия, совпадающие с реализацией ноутбука:

- существует ровно одна вершина Input;
- существует ровно одна вершина Output;
- каждое ребро ссылается только на существующие вершины;
- граф ациклический;
- параметры слоев допустимы;
- проверки форм тензоров для Add, Concat, Linear и пространственных операций выполняются.

Предикат корректности обозначим как

$$\mathcal{V}(G). \quad (3)$$

2.2 Каноническая форма

Определение 2. Для каждого корректного графа G его каноническая форма $C(G)$ строится следующим образом:

1. вершины сортируются по стабильным идентификаторам;
2. ребра сортируются лексикографически по паре ("src", "dst");
3. отсутствующие словари параметров заменяются пустыми словарями;
4. словари сериализуются с сортировкой ключей.

Назначение канонизации состоит в удалении всех различий, вызванных только порядком записи объекта, а не его архитектурным смыслом.

2.3 Отображения кодирования

Пусть:

- J — детерминированная JSON-сериализация;
- Z — lossless-сжатие zlib;
- I — преобразование непустой последовательности байтов в положительное целое число;
- H — SHA-256 checksum, также переведенная в целое число.

Тогда определим

$$p(G) = I(Z(J(C(G)))). \quad (4)$$

$$h(G) = H(Z(J(C(G)))). \quad (5)$$

Итоговое кодирование имеет вид

$$\mathcal{E}(G) = [m, v, p(G), h(G)], \quad (6)$$

где m и v — заранее фиксированные константы формата Magic и Version.

Процедура декодирования выполняет обратные преобразования в обратном порядке, а затем снова применяет канонизацию и валидацию.

3. ОСНОВНЫЕ РЕЗУЛЬТАТЫ

3.1 Детерминированность канонизации

Theorem 1. Если два корректных графа G_1 и G_2 описывают одну и ту же архитектуру и отличаются только порядком вершин, ребер или порядком ключей в словарях, то

$$C(G_1) = C(G_2). \quad (7)$$

Proof. Канонизация использует только детерминированные операции:

- сортировку вершин по идентификатору;
- сортировку ребер по упорядоченной паре ("src", "dst");
- вставку одинакового пустого словаря параметров там, где он отсутствует;
- сериализацию с фиксированным порядком ключей.

Если G_1 и G_2 содержат одну и ту же архитектурную информацию, то после удаления различий, связанных только с порядком записи, нормализованные списки вершин, ребер и параметров совпадут. Следовательно,

$$C(G_1) = C(G_2). \quad (8)$$

□

3.2 Обратимость перехода «байты -> целое число»

Lemma 1. Пусть B — непустая последовательность байтов. Пусть $I(B)$ — перевод B в целое число по основанию 256, а $B'(I(B))$ — восстановление минимальной байтовой записи этого числа. Тогда

$$B'(I(B)) = B. \quad (9)$$

Proof. Отображение I является стандартной биекцией между непустыми конечными последовательностями байтов и положительными целыми числами, записанными в системе счисления по основанию 256 без лишних ведущих нулевых блоков. Реализация восстанавливает длину байтовой последовательности по битовой длине числа и тем самым возвращает единственное минимальное байтовое представление. Следовательно, переход к целому числу и обратный переход сохраняют исходную последовательность байтов. □

3.3 Точное round-trip восстановление

Theorem 2. Для любого корректного графа G выполняется

$$\mathcal{D}(\mathcal{E}(G)) = C(G). \quad (10)$$

Proof. По определению,

$$\mathcal{E}(G) = [m, v, I(Z(J(C(G))))], H(Z(J(C(G))))]. \quad (11)$$

При декодировании сначала проверяются первые две компоненты. Так как вектор получен из $\mathcal{E}(G)$, эти проверки проходят успешно.

По предыдущей лемме, обратное преобразование целого числа в байты восстанавливает точно строку

$$Z(J(C(G))). \quad (12)$$

Проверка контрольной суммы также проходит, поскольку checksum была вычислена именно от этой же сжатой байтовой строки:

$$H(Z(J(C(G)))) = h(G). \quad (13)$$

Так как zlib-сжатие обратимо, получаем

$$Z^{-1}(Z(J(C(G)))) = J(C(G)). \quad (14)$$

Так как JSON-десериализация является обратной к JSON-сериализации на сериализованном объекте, получаем

$$J^{-1}(J(C(G))) = C(G). \quad (15)$$

После этого декодер снова применяет канонизацию и валидацию. Канонизация не меняет уже канонический объект, а валидация проходит успешно, потому что исходный граф был корректен и никакая информация не была потеряна. Следовательно,

$$\mathcal{D}(\mathcal{E}(G)) = C(G). \quad (16)$$

□

3.4 Корректность процедуры декодирования

Theorem 3. Если процедура $\mathcal{D}(z)$ завершилась успешно и вернула граф G , то G является корректным каноническим графом архитектуры.

Proof. Успешное завершение декодирования означает, что были успешно пройдены все проверки:

1. вектор имел ровно четыре компоненты;
2. сигнатура формата была корректной;
3. версия схемы поддерживалась;
4. payload удалось перевести обратно в байты;
5. checksum совпала;
6. разжатие выполнилось без ошибки;
7. JSON был успешно разобран;
8. канонизация выполнялась;
9. validate_graph не выбросила исключение.

Последний пункт в точности означает, что выполнен предикат $\mathcal{V}(G)$. Поскольку перед возвратом применяется канонизация, возвращаемый граф также является каноническим. Следовательно, G — корректный канонический граф архитектуры. □

3.5 Невозможность точного кодирования в конечный fixed-bit вектор

Proposition 1. Не существует инъективного точного кодирования всех поддерживаемых архитектур CNN в вектор фиксированной длины, если каждая координата такого вектора имеет фиксированное конечное число бит.

Proof. Пусть длина вектора равна d , а каждая координата содержит ровно b бит. Тогда число различных таких векторов конечно и равно

$$N = 2^{db}. \quad (17)$$

Теперь рассмотрим бесконечное семейство корректных последовательных архитектур

$$G_n = \text{Input} \rightarrow \text{Conv}_1 \rightarrow \text{Conv}_2 \rightarrow \dots \rightarrow \text{Conv}_n \rightarrow \text{Output}, \quad (18)$$

где $n \geq 1$, а каждый узел "Conv"_i — допустимый сверточный слой с фиксированным набором параметров. При разных n графы G_n различны, следовательно, множество поддерживаемых архитектур бесконечно.

Инъективное отображение бесконечного множества в конечное множество не существует. Получаем противоречие. Следовательно, точное кодирование всех поддерживаемых архитектур в конечный fixed-bit вектор невозможно. □

3.6 Почему вектор из четырех целых чисел не противоречит этому результату

Corollary 1. Представление

$$[m, v, p, h] \in \mathbb{Z}^4 \quad (19)$$

не противоречит предыдущему утверждению о невозможности.

Proof. Предыдущее утверждение относится к векторам, у которых фиксированы и длина, и конечная битовая ширина каждой координаты. В реализованном представлении фиксирована только длина. Компоненты же являются математическими целыми числами неограниченной разрядности. Множество $\mathbb{Z}^4$ бесконечно, поэтому здесь нет кардинального противоречия с точным кодированием бесконечного множества конечных графов. □

3.7 Поддержка skip connections и ветвлений

Theorem 4. Используемая графовая модель точно представляет skip connections типа ResNet и ветвления/слияния типа Inception.

Proof. Любая архитектурная зависимость кодируется явным ориентированным ребром из множества E .

Для ResNet-подобной skip connection пусть u — ранний узел, а a — узел типа Add. Тогда shortcut-путь задается дополнительным ребром

$$(u, a) \in E. \quad (20)$$

Основная ветвь дает одно или несколько дополнительных входящих ребер в тот же узел Add.

Для Inception-подобного блока пусть $b_1, b_2, \dots, b_k$ — финальные узлы параллельных ветвей, а c — узел типа Concat. Тогда операция слияния задается ребрами

$$(b_i, c) \in E, \quad i = 1, \dots, k. \quad (21)$$

Так как все эти ребра хранятся явно, информация о shortcut-путях и параллельных ветвях не теряется. Следовательно, оба паттерна представляются точно. $\square$

4. ЗАКЛЮЧЕНИЕ

Используемое в лабораторной работе 5 представление можно описать как композицию прозрачных математических отображений:

$$\begin{aligned} G &\rightarrow C(G) \\ &\rightarrow J(C(G)) \\ &\rightarrow Z(J(C(G))) \\ &\rightarrow I(Z(J(C(G)))). \end{aligned} \quad (22)$$

Обратное направление корректно определено на тех векторах, которые проходят проверки формата, checksum и структурной корректности. Следовательно, для любого корректного поддерживаемого графа архитектуры G выполняется

$$\mathcal{D}(\mathcal{E}(G)) = C(G). \quad (23)$$

Кроме того, любой успешно декодированный объект гарантированно является корректным каноническим графом. Это и составляет полное формальное обоснование использованного в лабораторной работе фиксированного векторного представления архитектур CNN.